

10.3.5 工程版源码走读：跟随 + 仲裁全景串讲

这份文档怎么用

- 1. 《10.3.5_多线程推理与视觉跟随控制》——原理 + LAB + 图；
- 2. 本文——走读：对着仓库真实文件逐段看「为什么这样写」。

推荐先读讲义到 § 2.2（跟踪原理）并做过 LAB1，再打开编辑器对照本文。
行号以当前仓库为准；若你本地有改动，以函数名为准跳转（Ctrl+P / Ctrl+点击）。

第 0 章 源码地图（先收藏这一页）

文件	角色	讲义对应	必读符号
src/mecamind_bringup/launch/follow.launch.py	总装：仿真+眼睛+跟随+仲裁+安全门	LAB / § 2.8	generate_launch_description
src/mecamind_perception/.../detector_node.py	双线程 + LatestFrameQueue	§ 2.1	LatestFrameQueue / _inference_loop
src/mecamind_tools/.../perception_filter.py	连续帧确认	§ 2.2	select_detection / _cb
src/mecamind_tools/.../vision_follow_controller.py	P 控制律 → /cmd_vel_follow	§ 2.2	compute_follow_command
src/mecamind_perception/.../cmd_vel_arbiter.py	速度仲裁	§ 2.3	pick_active_source / _tick
src/mecamind_tools/.../safety_layer.py	安全门（可执行名 mecamind_safety_gate）	§ 2.4	analyze_scan / gate_twist
src/mecamind_tools/.../follow_target_mover.py	Gazebo 红柱绕圈	§ 2.5	_on_timer / _set_pose
src/mecamind_gazebo/worlds/three_room_house_follow.sdf	跟随专用出生位姿	§ 2.5	mecamind_mecanum / follow_target
scripts/test_cp5.sh	CP5 自动考官	§ 8	全文

0.1 数据流（节点名 ↔ 文件）

```
mecamind_detector          ← detector_node.py
  → /mecamind/detections
mecamind_detection_json_bridge ← json_bridge_node.py（第四节）
  → /mecamind/raw_detections
mecamind_perception_filter   ← perception_filter.py
  → /mecamind/perception_event
mecamind_vision_follow_controller ← vision_follow_controller.py
  → /cmd_vel_follow
mecamind_cmd_vel_arbiter     ← cmd_vel_arbiter.py
  → /cmd_vel_nav
mecamind_follow_safety_gate  ← safety_layer.py
  → /cmd_vel

可选：mecamind_follow_target_mover ← follow_target_mover.py
```

0.2 建议走读顺序

launch 骨架 → 队列解耦 → 过滤确认 → P 控制 → 仲裁 → 安全门 → 红柱演示 → CP5。

每章结构：打开哪个文件 → 先读哪段注释 → 精读哪几行 → 走读自查。

第 1 章 follow.launch.py：一条命令如何拼起来

打开： `src/mecamind_bringup/launch/follow.launch.py`

1.1 先读文件头（比函数体更重要）

文件顶部 docstring 已经画出两条链路：

```
视觉：detector → detections → json_bridge → filter → perception_event → follow → cmd_vel_follow
速度：teleop / follow / nav_in → arbiter → cmd_vel_nav → safety_gate → cmd_vel
```

课上改演示参数时，**优先改本文件参数块**，不要先改节点默认值——launch 才是「课堂配置面」。

1.2 generate_launch_description 积木表

Launch 积木	对应节点/动作	课堂常改参数
gazebo_backend / lite_backend	仿真	backend / use_gui / world
detector + json_bridge + perception_filter	眼睛	source / infer_backend / target_label
follow_target_mover	红柱（条件： gazebo+moving_target）	waypoints_xy / speed_mps
follow_controller	P 跟随	desired_width / max_linear
auto_enable_follow	~10s 后连发使能	TimerAction.period
arbiter	仲裁	一般不改
safety_gate	安全门	与第三节同款接线

1.3 走读自查

- 1. 为什么跟随输出话题是 /cmd_vel_follow 而不是 /cmd_vel？
- 2. moving_target:=true 在 lite 后端会不会启动红柱节点？

第 2 章 LatestFrameQueue：容量 1 为什么是优点

打开： src/mecamind_perception/mecamind_perception/detector_node.py

2.1 先读模块 docstring（约第 1-39 行）

把「采集 → 容量 1 队列 → 推理线程」这张图画在纸上，再往下看代码。

2.2 精读 put_latest（约第 90-100 行）

```
def put_latest(self, packet: FramePacket) -> None:
    while True:
        try:
            self._q.put_nowait(packet)
            return
        except queue.Full:
            try:
                self._q.get_nowait()
                self.dropped += 1
            except queue.Empty:
                pass
```

读法：

- 队列 maxsize=1（见 __init__）；
- 放不进去 → 先扔掉槽里那一帧（旧帧）→ dropped += 1 → 再放新帧；
- 所以控制系统永远更接近「现在」，代价是中间帧可以丢。

2.3 再搜这两个方法

符号	读什么
<code>_capture_loop</code>	视频/合成源如何周期 <code>put_latest</code>
<code>_inference_loop</code>	如何 <code>get</code> → <code>detect</code> → 发布 <code>detections / metrics</code>

2.4 走读自查

- 1. 队列满时丢掉的是旧帧还是新帧？
- 2. `dropped_frames` 上涨时，车看到的世界是更新了还是更旧了？

第 3 章 `compute_follow_command`: 本课全部跟踪原理

打开: `src/mecamind_tools/mecamind_tools/vision_follow_controller.py`

3.1 先读文件头「三层保护」

使能开关 / `confirmed` 门槛 / 看门狗超时——比增益更重要。

3.2 精读控制律核心（约第 89–112 行）

```
error_x = cx - 0.5
angular = 0.0 if abs(error_x) < center_deadband else -kp_angular * error_x
angular = max(-max_angular, min(max_angular, angular))

if target_width <= 0.0:
    return FollowCommand(0.0, angular, True, "no_width")

proximity = target_width / max(1e-3, desired_width)
if proximity < 1.0:
    # 越远 (proximity 越小) 越快; kp_linear 放大追赶积极性
    linear = max_linear * (1.0 - proximity) * max(0.5, kp_linear)
    linear = min(max_linear, linear)
else:
    # 过近: 明显后退, 避免顶到/穿进目标
    overshoot = min(2.0, proximity - 1.0)
    linear = -max_linear * 0.95 * min(1.0, overshoot / 0.6)

# 大转向时压线速度, 先对准再加速
turn = abs(error_x)
if turn > center_deadband:
    turn_scale = max(0.25, 1.0 - (turn - center_deadband) / 0.40)
    linear *= turn_scale

return FollowCommand(linear, angular, True, "target_confirmed")
```

三个「为什么」:

写法	为什么
<code>-kp_angular * error_x</code>	图像右偏为正，ROS 左转也为正，必须取反才「朝目标转」
<code>desired_width</code> 作分母	目标要在画面里「看起来多大」才算合适距离； 越大跟越近
过近用接近满速后退	课堂演示防穿模；不是完整 PID 的 D 项

3.3 节点侧三个回调（同文件）

符号	约略行号	读什么
<code>should_publish_follow</code>	~115	双条件闸门
<code>_enable_cb</code>	~159	关闭时 主动 发零速
<code>_event_cb</code>	~176	JSON 事件 → Twist
<code>_watchdog</code>	~209	丢目标刹停 / 慢搜

3.4 走读自查

1. 为什么 `desired_width` 越大反而跟得越近？
2. 只停发消息、不发零速 Twist，底盘可能怎样？

第 4 章 perception_filter：别让误检开车

打开： `src/mecamind_tools/mecamind_tools/perception_filter.py`

符号	作用
<code>parse_detections</code>	宽容解析 JSON（字段名不统一也能吃）
<code>select_detection</code>	按 label + 置信度挑一个
<code>PerceptionFilterNode._cb</code>	<code>confirm_frames</code> 连续确认去抖

跟随控制器只订阅 `/mecamind/perception_event`，不直接吃 `/mecamind/detections` ——防火墙。

走读自查

把 `confirm_frames` 从 3 改成 1，跟随会更「跟手」还是更「抽风」？为什么？

第 5 章 cmd_vel_arbiter：谁说了算

打开： `src/mecamind_perception/mecamind_perception/cmd_vel_arbiter.py`

5.1 优先级表（约第 48 行）

```
SOURCE_PRIORITY = ("teleop", "follow", "nav")
```

急停不在这张表里——单独锁存，高于一切。

5.2 pick_active_source (约第 59–73 行)

纯函数：按表顺序找第一个「时间戳仍新鲜」的源。
单元测试可以穷举，不必起 Gazebo。

5.3 _estop_cb / _tick

行为	代码意图
estop=True	立即发零速，并锁存
抢占切换	先发一帧零速，再转发新源
全员沉默	只在刚变沉默时刹一帧，然后闭嘴

走读自查

遥控话题若持续以 0 速刷屏，跟随还能拿回控制权吗？为什么？

第 6 章 safety_layer：听了之后还安不安全

打开： `src/mecamind_tools/mecamind_tools/safety_layer.py`
入口名： `mecamind_safety_gate` (`setup.py` `entry_points`)

符号	读什么
文件头 docstring	在链路中的位置
<code>analyze_scan</code>	前/后/左/右扇区 + 停/警告线
<code>gate_twist</code>	如何改写 Twist (含 <code>linear.y</code> 横移)

仲裁管「听谁的」；本文件管「前方有墙能不能照做」。两层缺一不可。

第 7 章 follow_target_mover：演示用红柱

打开： `src/mecamind_tools/mecamind_tools/follow_target_mover.py`

符号	读什么
模块 docstring	为何绕圈、默认几何
<code>__init__</code> 里 <code>closed_loop</code>	首尾闭合才能 <code>fmod</code> 绕圈
<code>_follow_enable_cb</code>	上升沿才重置，避免连发拽回起点
<code>_on_timer</code>	弧长 <code>s</code> 推进 + <code>set_pose</code>
<code>_set_pose</code>	<code>gz service .../set_pose</code>

世界出生点： `src/mecamind_gazebo/worlds/three_room_house_follow.sdf`
内含 `mecamind_mecanum` 与 `follow_target` 的 `<pose>`。

走读自查

若 `closed_loop=True` 但 waypoints 首尾不闭合、代码也忘了 `append` 起点，轨迹会怎样？

第 8 章 test_cp5.sh：自动考官查什么

打开： `scripts/test_cp5.sh`

检查点	对应讲义
角速度方向 vs synthetic 左右晃	§ 2.2 / LAB1
<code>follow_enable=false</code> 后停车	LAB2
teleop 抢占 / 交还	LAB3
estop 锁存	LAB3 / § 2.4

成功： `MECAMIND_CP5_OK`，报告 `maps/mecamind_cp5_report.json`。

第 9 章 推荐阅读路径（60 分钟自学）

分钟	做什么
0-10	对照讲义总框架图，在编辑器打开上表 6 个 <code>.py</code>
10-25	精读第 2+3 章（队列 + P 控制）
25-40	精读第 5 章仲裁 + 自己 <code>echo arbiter_state</code>
40-50	扫第 6-7 章，Gazebo 演示时对照日志
50-60	做文末自查；不会的回到讲义插图

走读自查（全部答得上再进第六节）

1. 指出从检测到 `/cmd_vel` 的每一个源文件。
2. 说明 `LatestFrameQueue` 容量为 1 时丢的是谁。
3. 在 `compute_follow_command` 里指出「负号」的几何含义。
4. 说明 `estop` 为什么不能靠「话题沉默」自动解除。
5. 说明红柱演示为何用 HSV 而不是 YOLO。
6. 说出改跟进/跟远时，应优先改 `launch` 里哪个参数。

与主讲义、当前 `follow.launch.py` / CP5 行为对齐（红柱南侧绕圈、`desired_width≈0.36`，跟踪原理为归一化 P 控制）。